



Exploring Semantic Embeddings

Introductory Topics: Text Embeddings, NumPy, Matplotlib,
Pickling, and Dimensionality Reduction with PCA and t-SNE

DSC 305

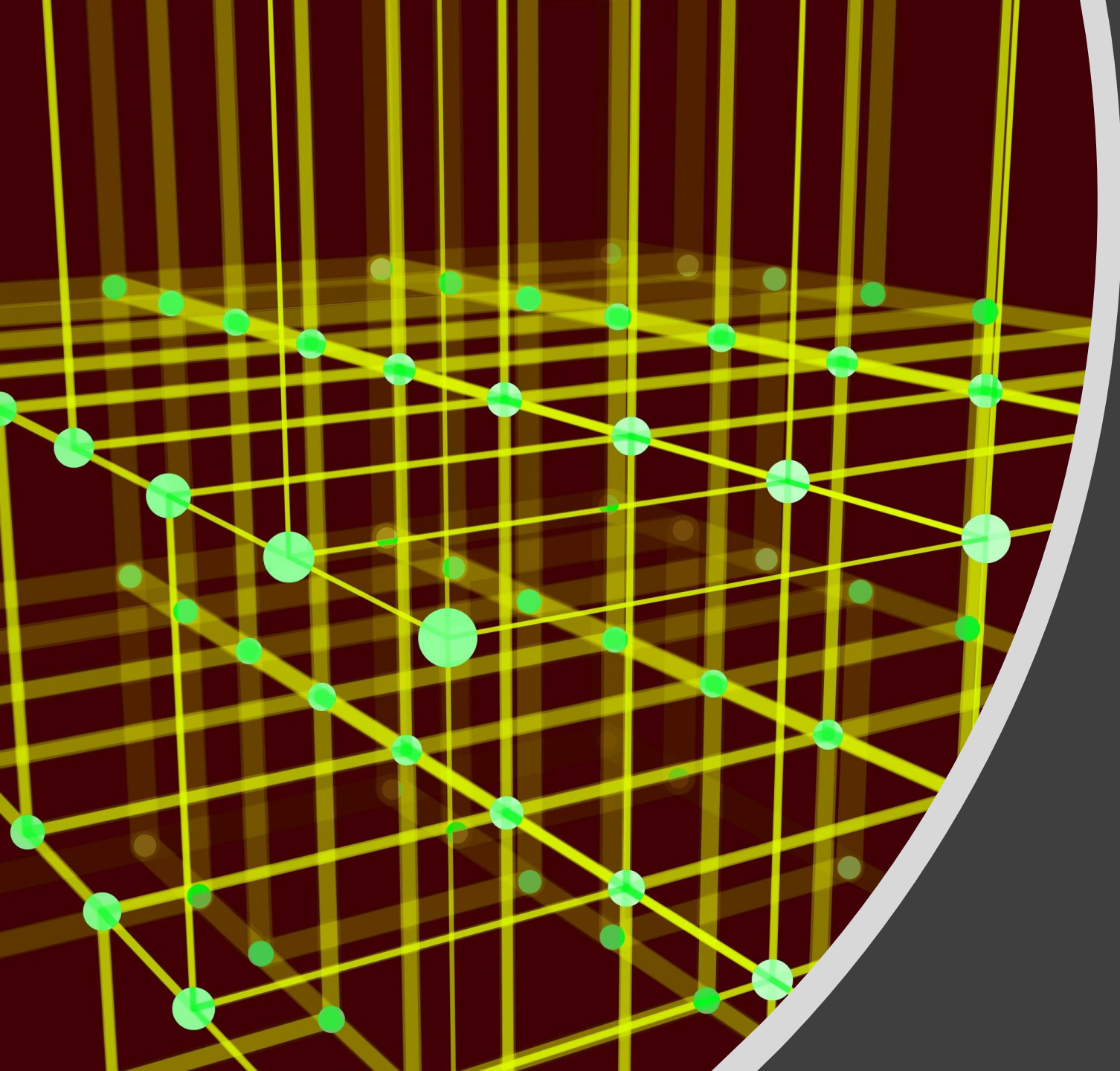
February 20, 2023

The Plan

- Last day of our introductory unit
- Text embeddings
- Feature space (high dimensional)
- Computing distance with NumPy
- Visualizing embeddings with Matplotlib
- Dimensionality reduction with PCA and t-SNE
- Finding neighbors for EDA, classification, and recommender systems

Announcements

- Quiz 2 on Wednesday
 - Olin 107 Lab
 - Covers everything through today
 - Study guide for QM-2 posted (draft)
 - Can also "level up" on QM-1
 - Work missed questions only
 - Similar, not identical, same numbers
- Continue working on Lab 2 with your partner
- New unit Friday: Tour of Models
 - Logistic regression



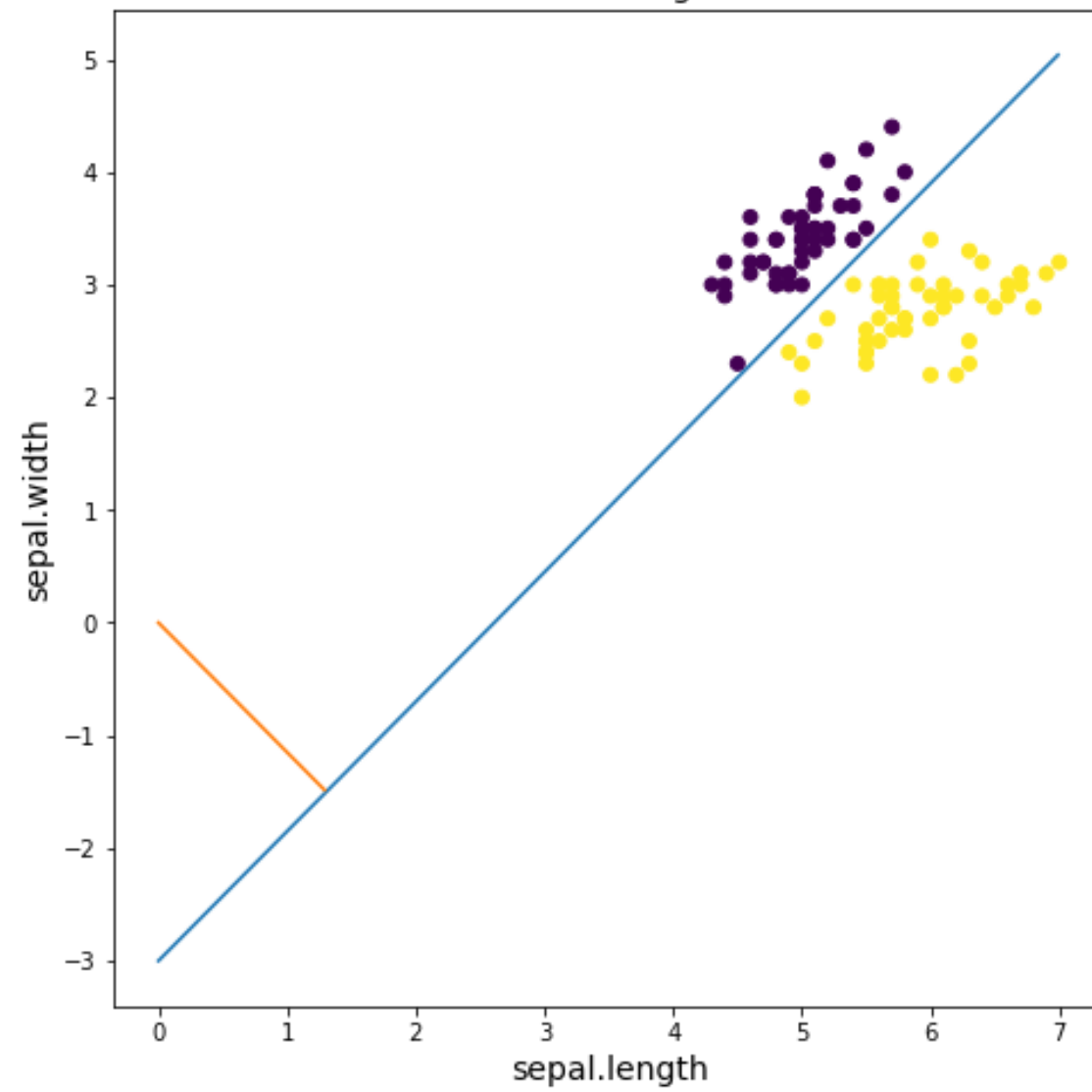
Feature Space

In ML we often embed data in high dimensional space

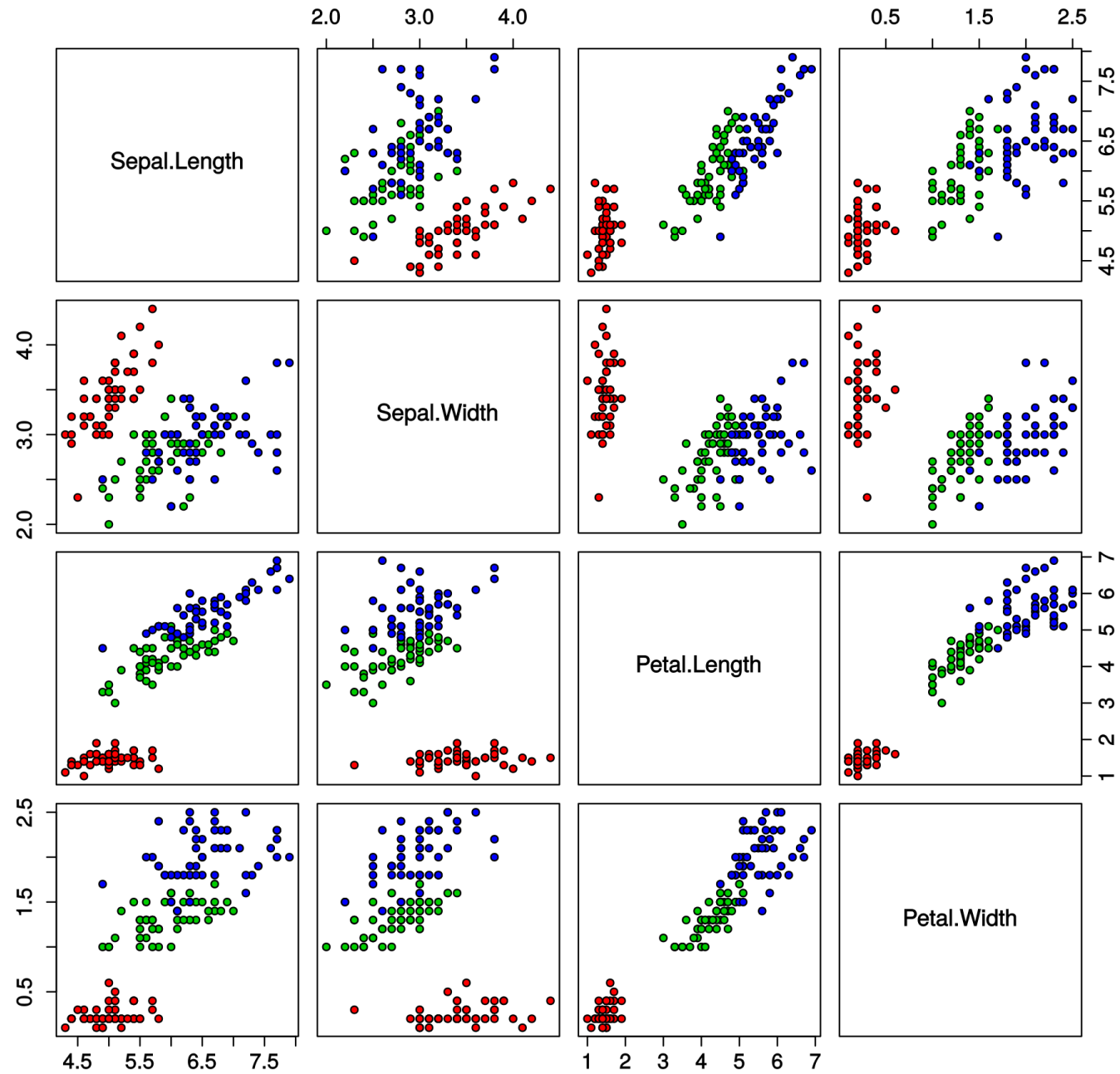


Iris Data Set

Linear model using Iris data

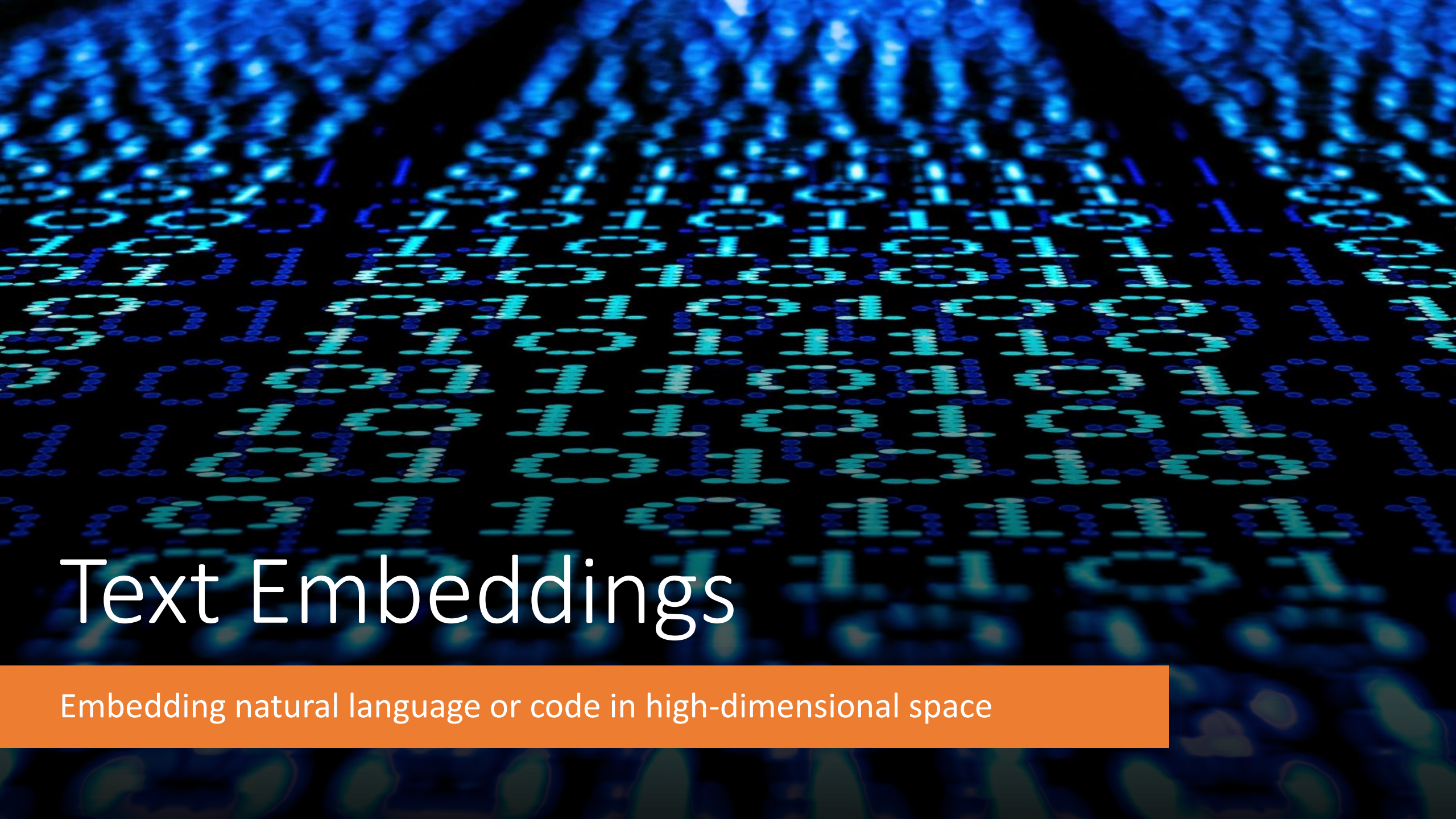


Iris Data (red=setosa,green=versicolor,blue=virginica)



Feature Space

- Important concept in ML.
- Multi-dimensional space.
- Data points as vectors of features.
- Gain insights into the data and make more accurate predictions.
- Linear algebra helpful but does not require deep understanding.
- Can also embed images, sound, and text in feature space.



Text Embeddings

Embedding natural language or code in high-dimensional space

Text Embeddings

- Transfer text into a numerical representation.
- Measure similarity between different words.
- Useful for Natural Language Processing (NLP).
- Can be used to classify text and generate recommendations.

Applications

- **Search** results ranked by relevance to a query
- **Clustering** text grouped by similarity
- **Recommendations** recommend items related to a query
- **Classification** classify text by instances with similar meaning
- **Anomaly detection** identify outliers
- **Diversity measurement** where similarity distributions are analyzed

Example

- Text: **The green elephant**
- Embedding: [-0.0139, -0.015 , 0.014 , -0.0121, -0.0050, ...]
- Space: 1536 dimensions
- Method: autoencoder trained on a large corpus of text
(OpenAI text-embedding-ada-002 model)

Obtaining embeddings from OpenAI API

Saving embeddings using Python Pickle tool

Coding Demo





Linear Algebra Basics

Using vectors to estimate distance

Dot product

- Calculation used to estimate similarity between 2 vectors.
- Multiplies corresponding entries and adds products together.

$$\mathbf{u} \cdot \mathbf{v} = \sum_{i=1}^n u_i v_i$$

- $u = [3.0, 4.0]$ $v = [11.0, 0.0]$. Then $u \cdot v =$ **33**
- $a = [2.0, 3.0, 1.0]$ $b = [4.0, 5.0, 2.0]$ Then $a \cdot b =$ **25**
- Hint: Make sure you can do this for the upcoming test!

Normalization

- **Normalization:** **rescale** vector so its **magnitude** (length) is 1 unit
- **Length of vector:** add square of every element, then take square root
- $u = [3, 4]$ $|u| = \text{sqrt}(3*3 + 4*4) = 5$
- $v = [11, 0]$ $|v| = \text{sqrt}(11*11 + 0*0) = 11$
- To normalize, divide all elements by length of vector
- $u = u / |u| = (1 / 5) * [3, 4] = [0.6, 0.8]$
- A **unit vector** is a vector with a length of 1.
- All vectors in our text embeddings dataset are unit vectors.

Vector Angle

- Formula:

$$\cos \theta = \hat{u} \cdot \hat{v} = \frac{u \cdot v}{\|u\| \|v\|}$$

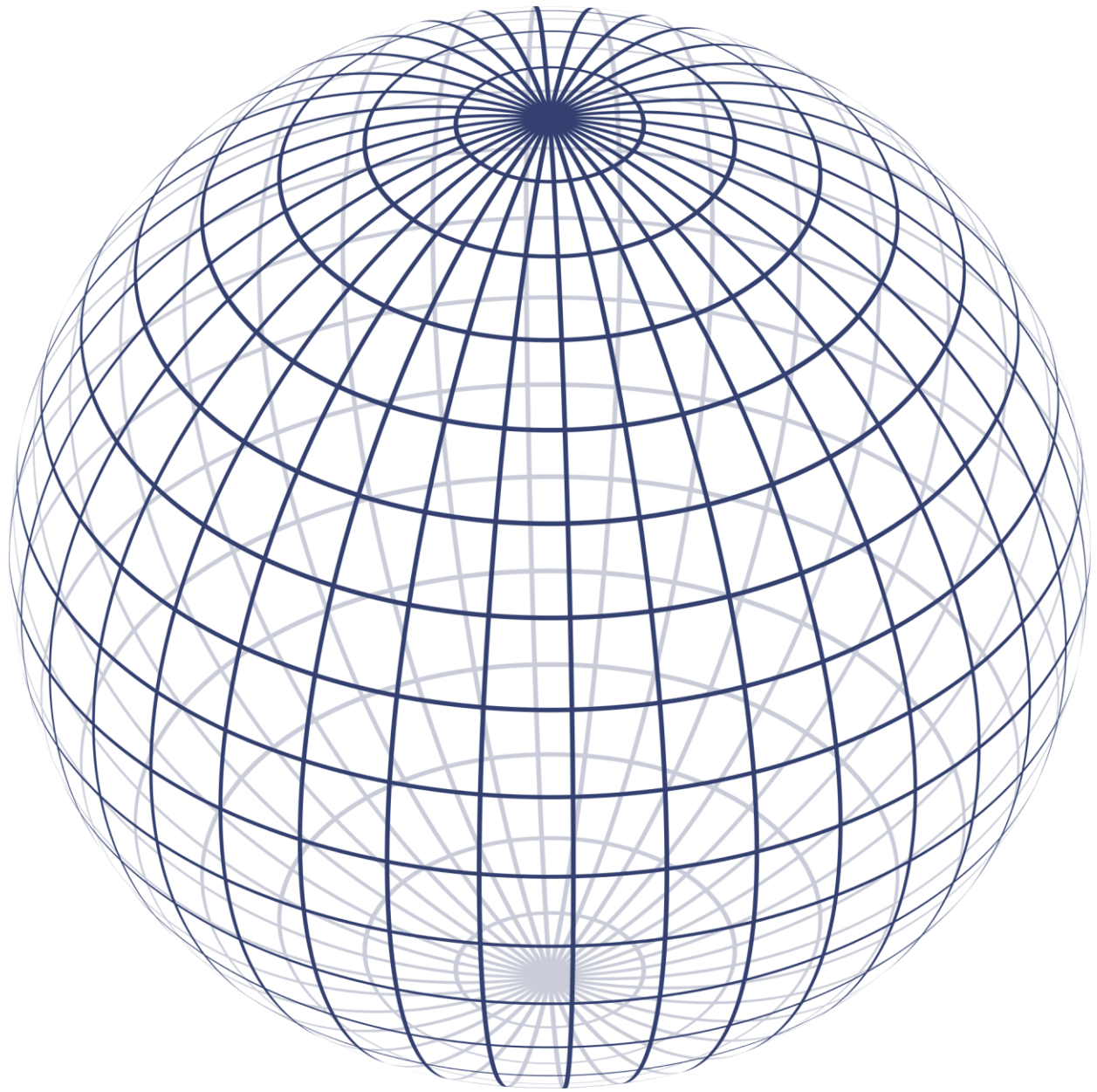
- To find the angle we would have to take the arc cosine.
- But if we just want to **compare** vectors, we just need the **dot product!**
- **The bigger the dot product, the more similar the data points!**
- Dot product of 1 means same point (most similar).
- Dot product of 0 means orthogonal. Can also be negative: [-1, 1]



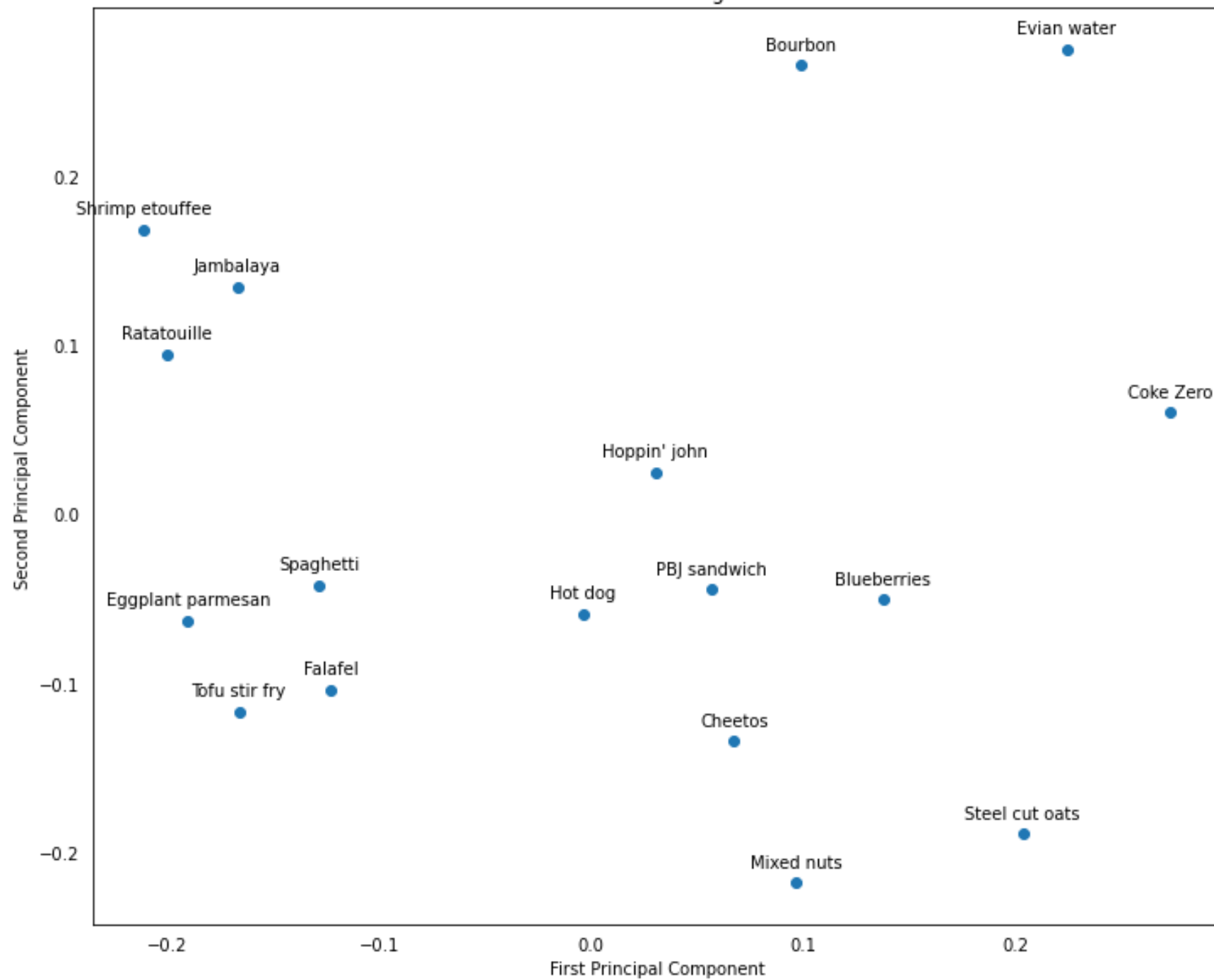
Visualizing Semantic Embeddings

Using Projections and Matplotlib

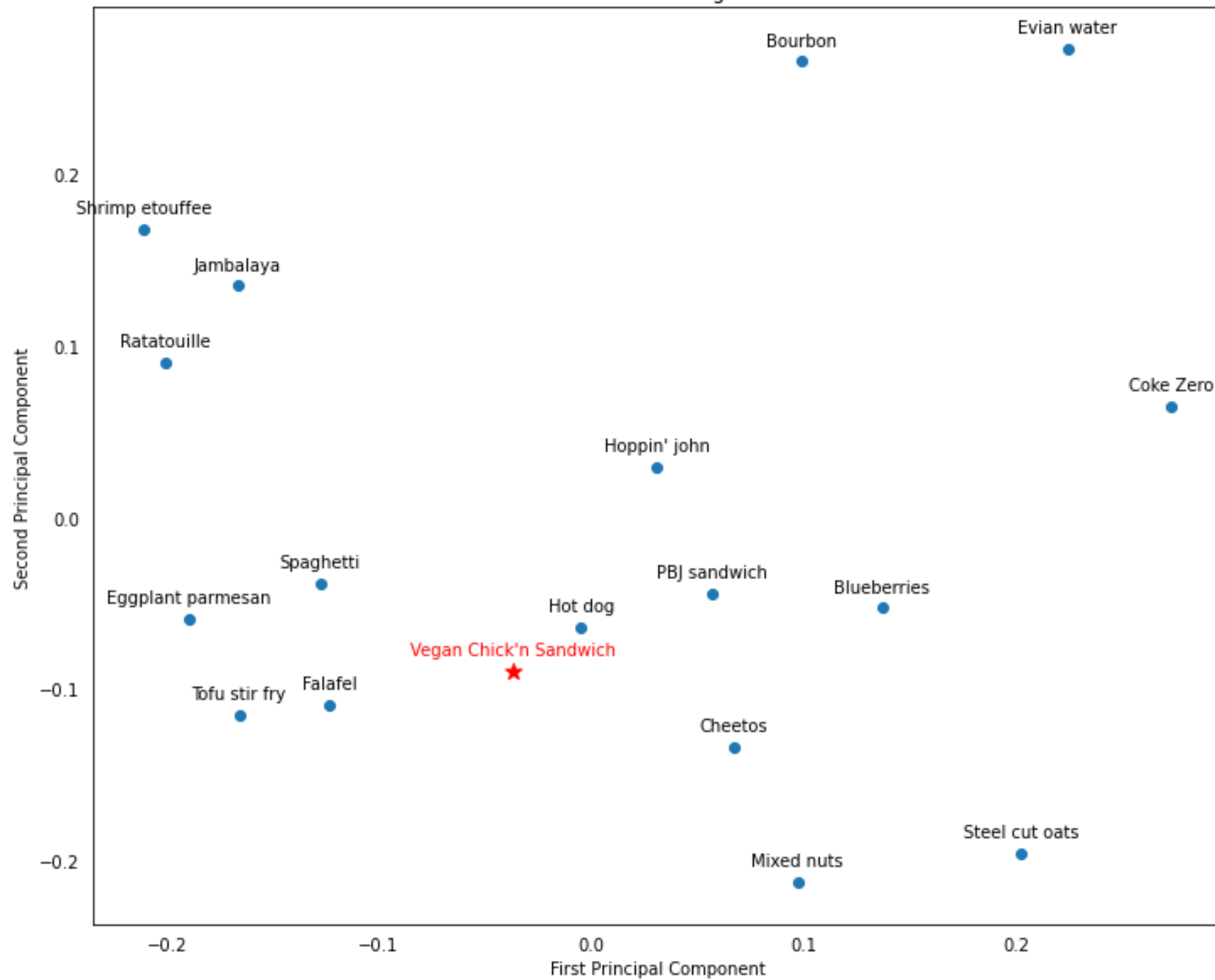
The background of the slide features a complex network visualization. It consists of numerous small, semi-transparent circular nodes connected by thin, light-colored lines. The nodes are colored in shades of teal, light blue, and dark grey. The network is dense and spans the entire width of the slide, with a slight gradient from dark on the left to light on the right.



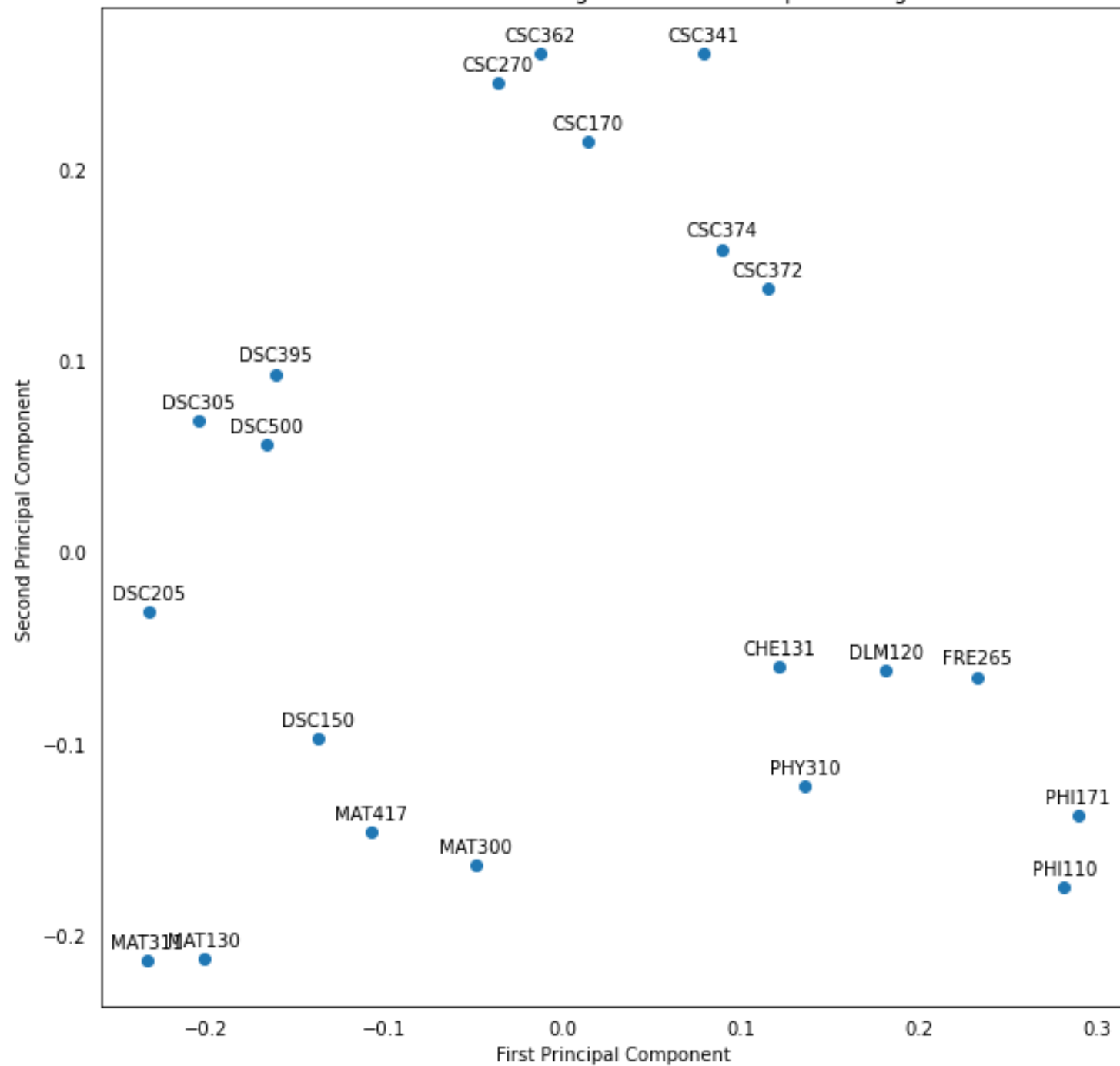
Food Semantic Embedding Visualization

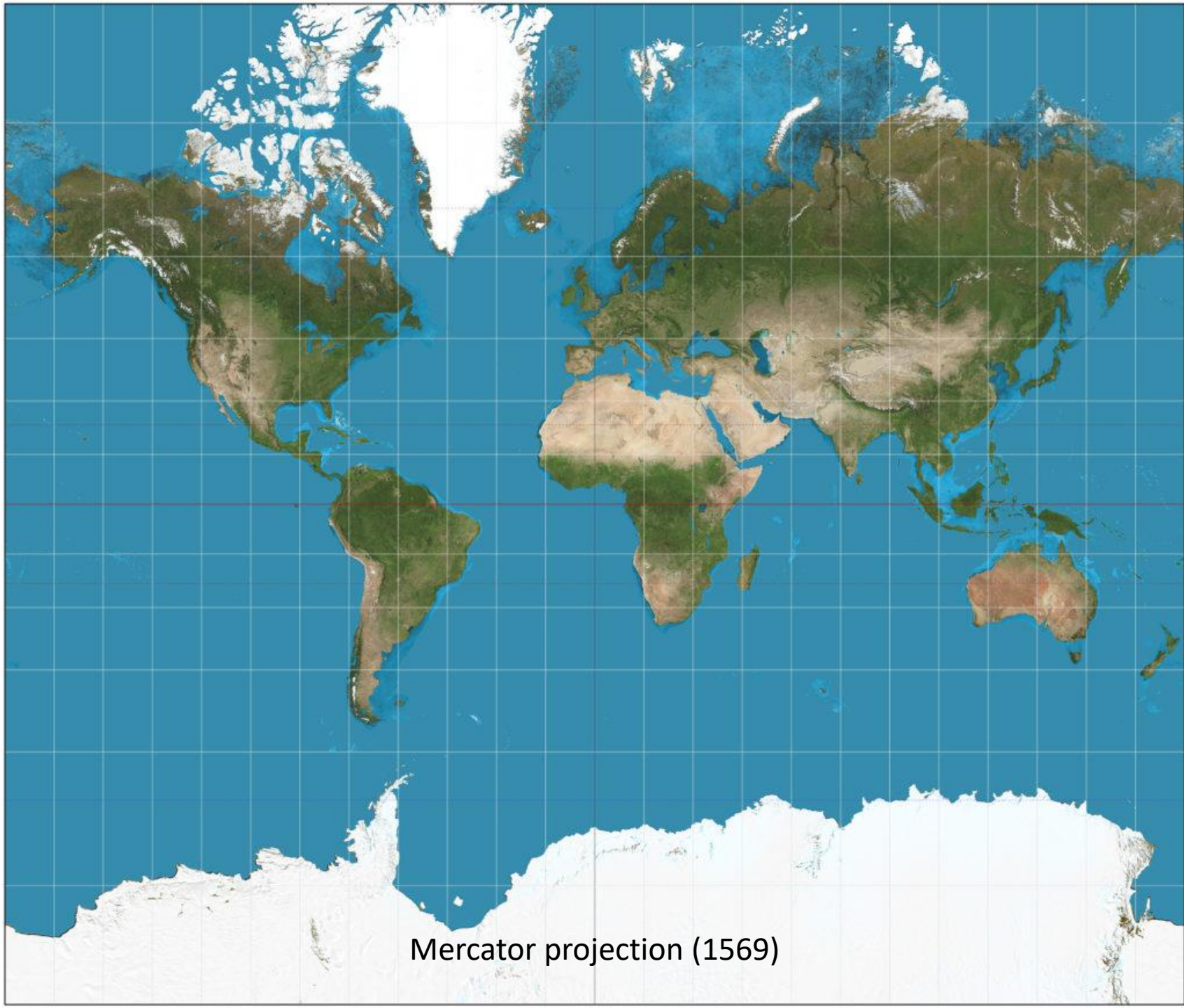


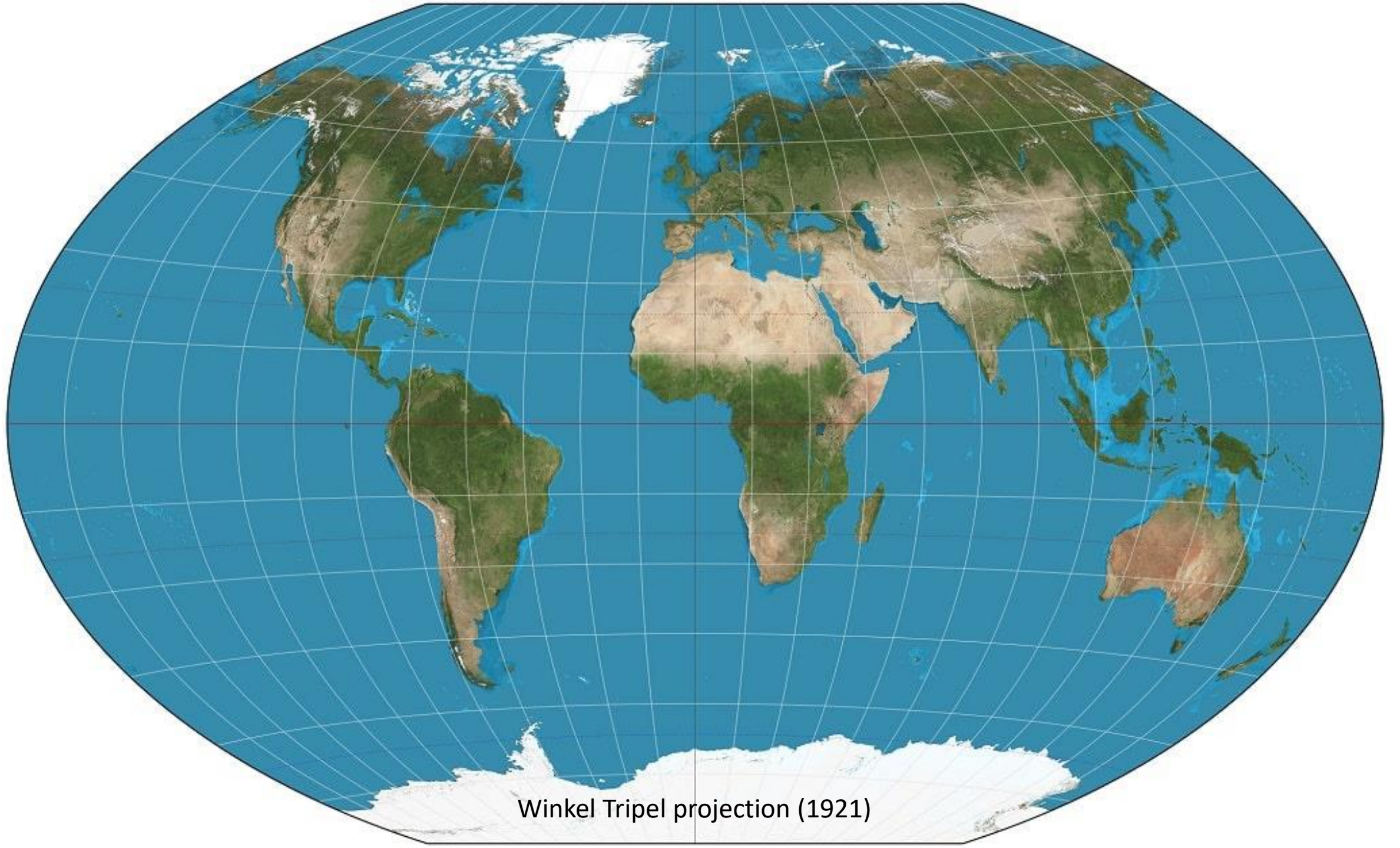
Food Semantic Embedding Visualization



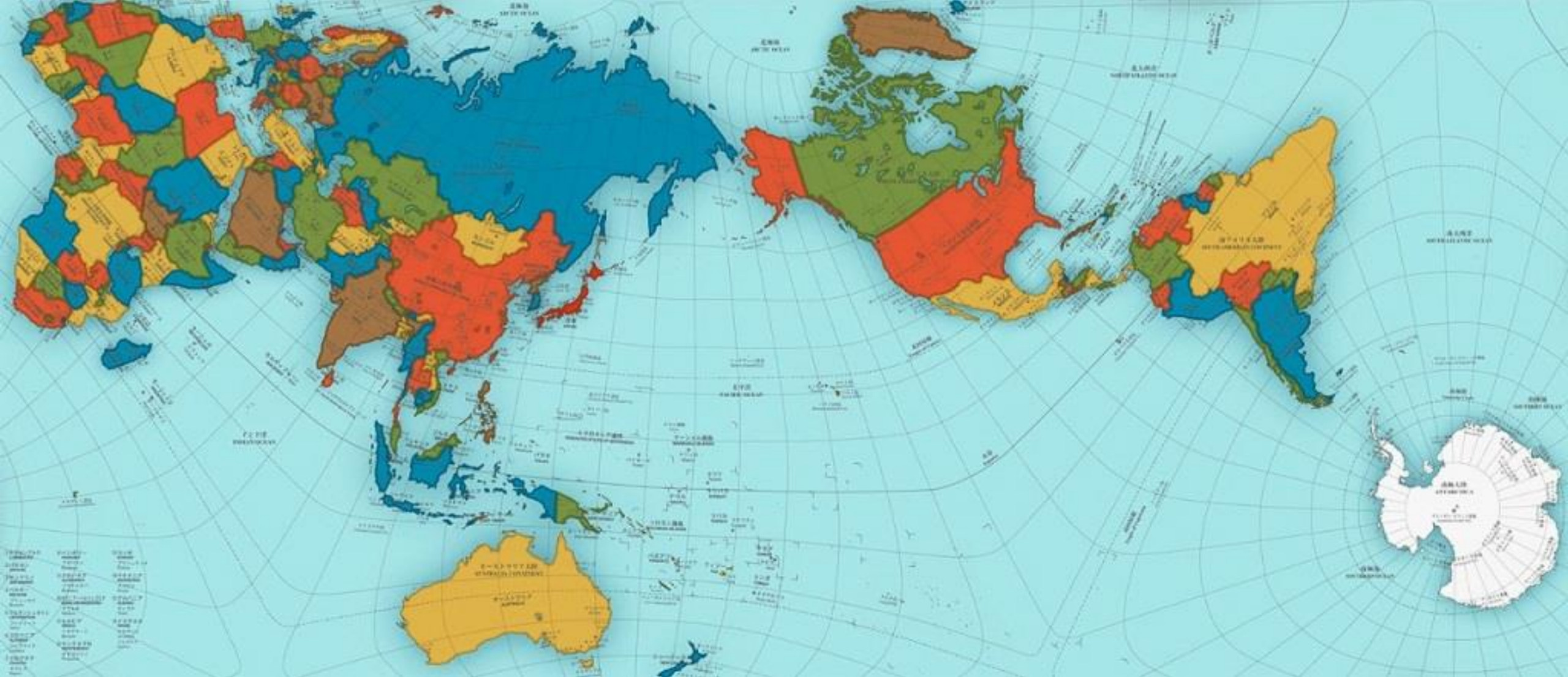
Visualization of Course Designation and Description using PCA







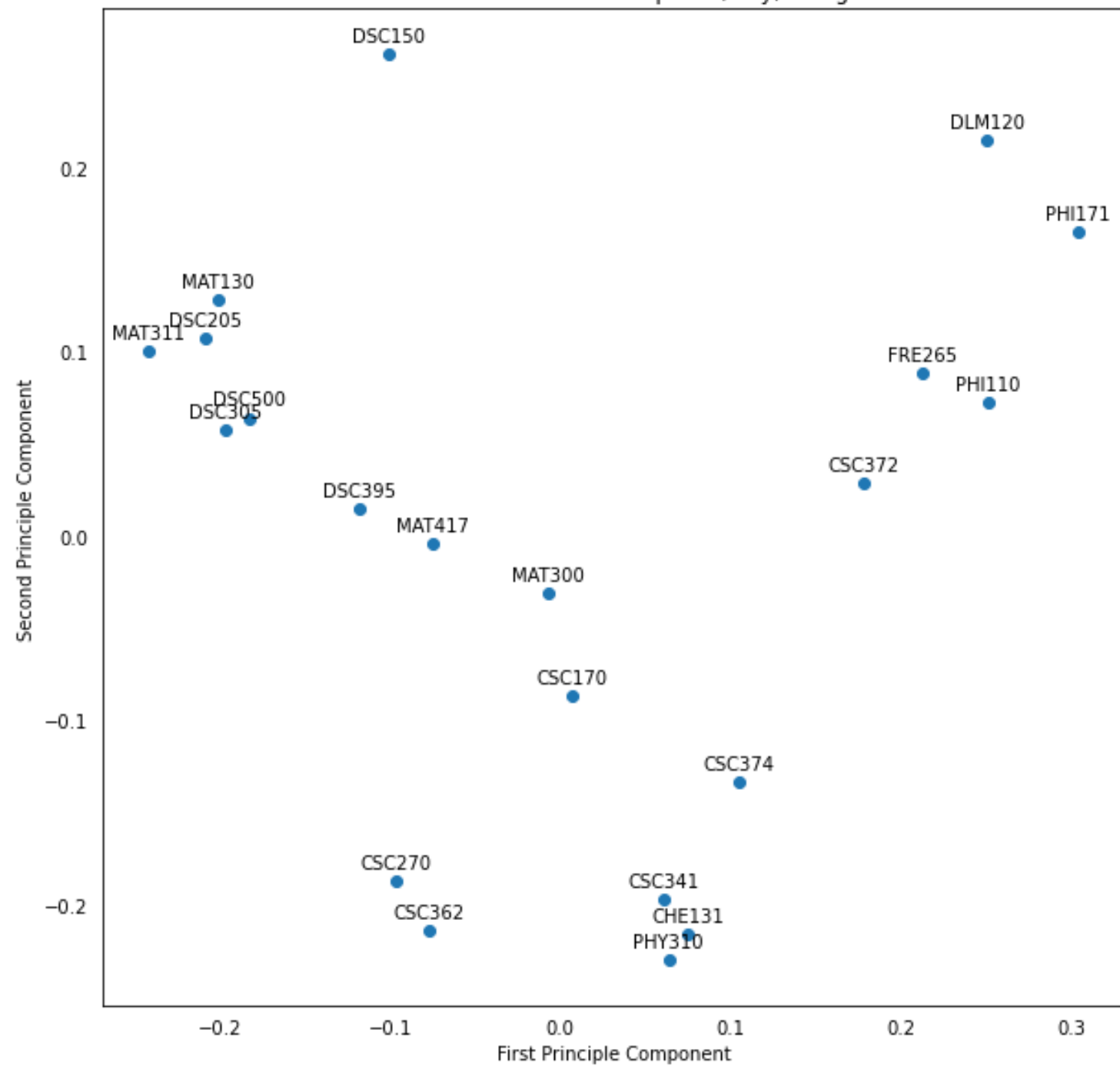
Winkel Tripel projection (1921)



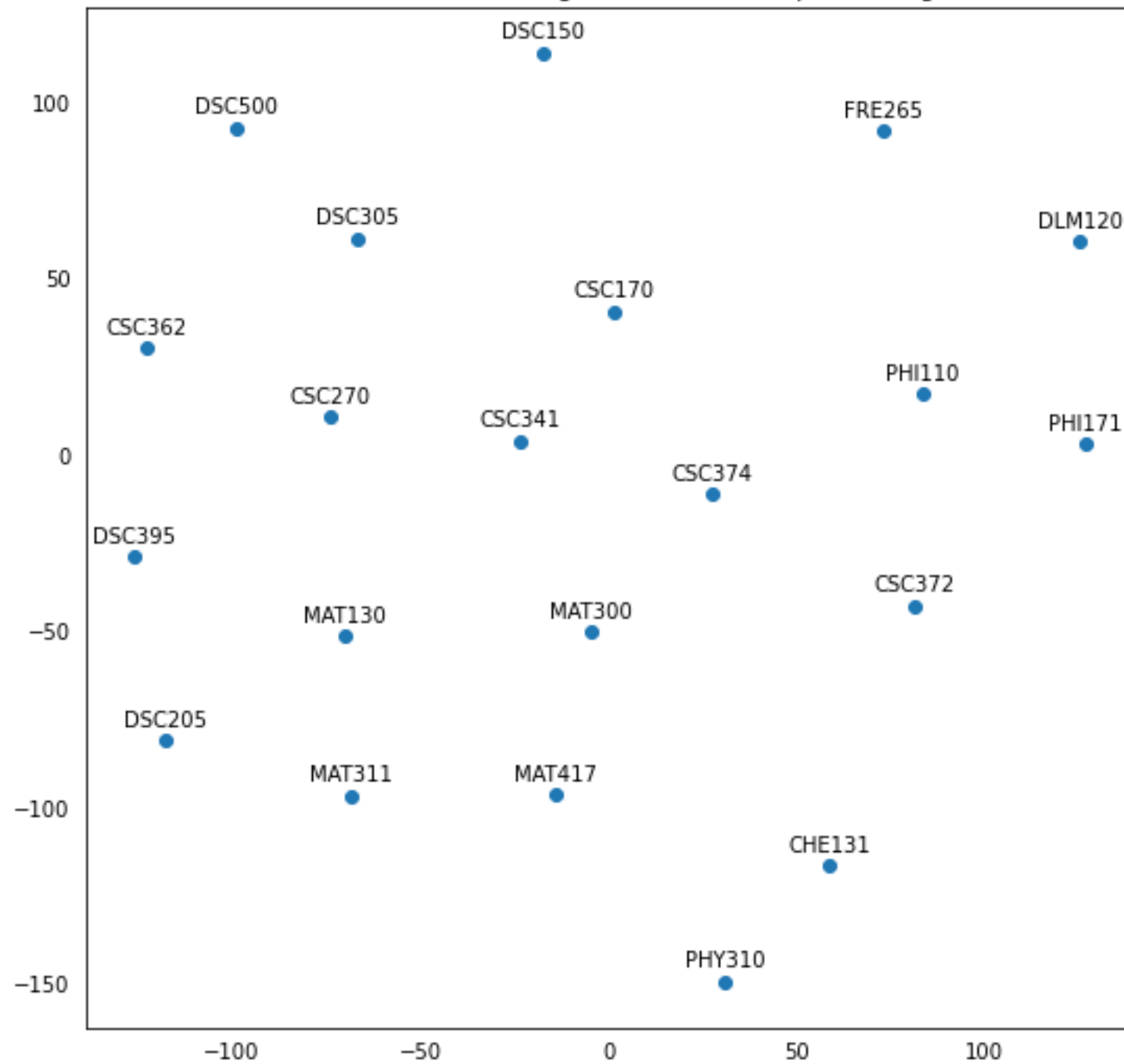
AuthaGraphy projection (Hajime Narukawa, 1999)

<https://repluglobes.com/blog/the-main-types-of-world-map-projections/>

Visualization of Course Description (only) using PCA



Visualization of Course Designation and Description using t-SNE



Projections

- Visualize high dimensional data in 2D or 3D
- Linear: Principal Component Analysis (PCA)
- Non-linear: t-Distributed Stochastic Neighbor Embedding (t-SNE)
- As with map projections, there are advantages and disadvantages
- Sadly, there are always some distortions
 - On previous slide, DSC395 is closest neighbor to CSC372 but looks far away.



Course Recommender System

Example Application

Recommender System

- Predict preferences or ratings that a user would give to an item
- Our example: given **user query**, recommend list of relevant courses
- Simple algorithm:
 - Let $u = \text{embedding}(\text{user_query})$
 - Let $v = \text{embedding}(\text{course_description})$
 - Then $u \cdot v$ denotes similarity of course to user interests
 - Choose the most similar k courses to present to the user
- Many other approaches, e.g., **collaborative filtering** (compare users)

What have we learned?

- Feature space is a central concept in ML
- Text embeddings: geometric distance approximates semantic distance
- If vectors are normalized, bigger dot product suggests more similarity
- Can study geometric relationships with NumPy linear algebraic library
- Can visualize with Matplotlib
- To visualize we first need to project onto 2- or 3-dimensional space
- Recommender system
- Reading check: EMC 1-2, PDSH 1-4 (can skim 2)